



EKS Cluster Building Blocks

Every EKS cluster has common building blocks: a managed control plane, worker nodes, a VPC, two IAM roles, and system addons. This lecture defines each and shows how they connect.

IN THIS LECTURE

01 Control plane

The managed API, plus the version, endpoint, and role you set.

02 Worker nodes

EC2 instances joined and replaced by a node group.

03 Networking

A VPC whose subnets hand pods real IP addresses.

04 IAM

Two service roles, plus AWS-identity access for operators.

05 Addons

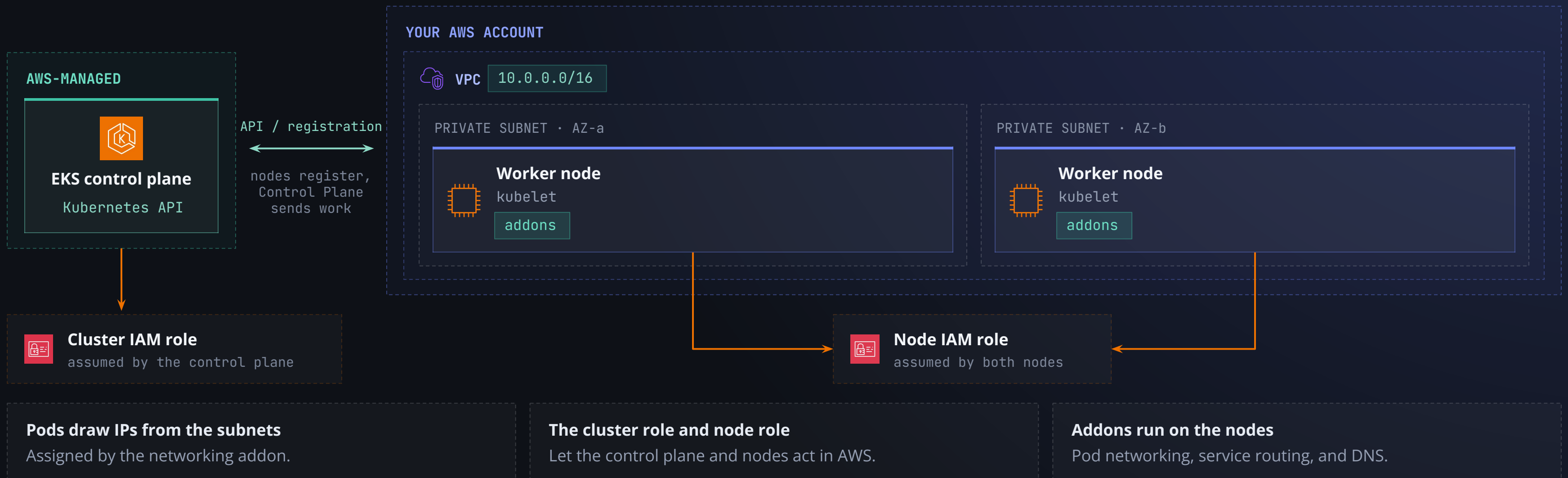
Pod networking, service routing, and in-cluster DNS.



THE WHOLE PICTURE

One cluster, five parts

AWS runs the control plane in its own account. Your nodes run in your VPC across two availability zones. A cluster role and a node role connect them to AWS, and addons on the nodes supply pod networking and DNS.



Control plane: version, endpoint, IAM role

AWS operates the API server, scheduler, controllers, and etcd. Your choices are the version it runs, whether its endpoint is reachable privately, publicly, or both, and the role EKS assumes to act in your account.



Kubernetes version

1.36

The minor version the control plane runs. It fixes API behavior and which addon versions are compatible.



Endpoint access

private | public | both

Whether the API server is reachable from inside the VPC, the public internet, or both. It decides who can attempt kubectl.



Cluster IAM role

assumed by EKS

The role EKS assumes to work in your account, for example creating load balancers to expose traffic from Kubernetes Services.

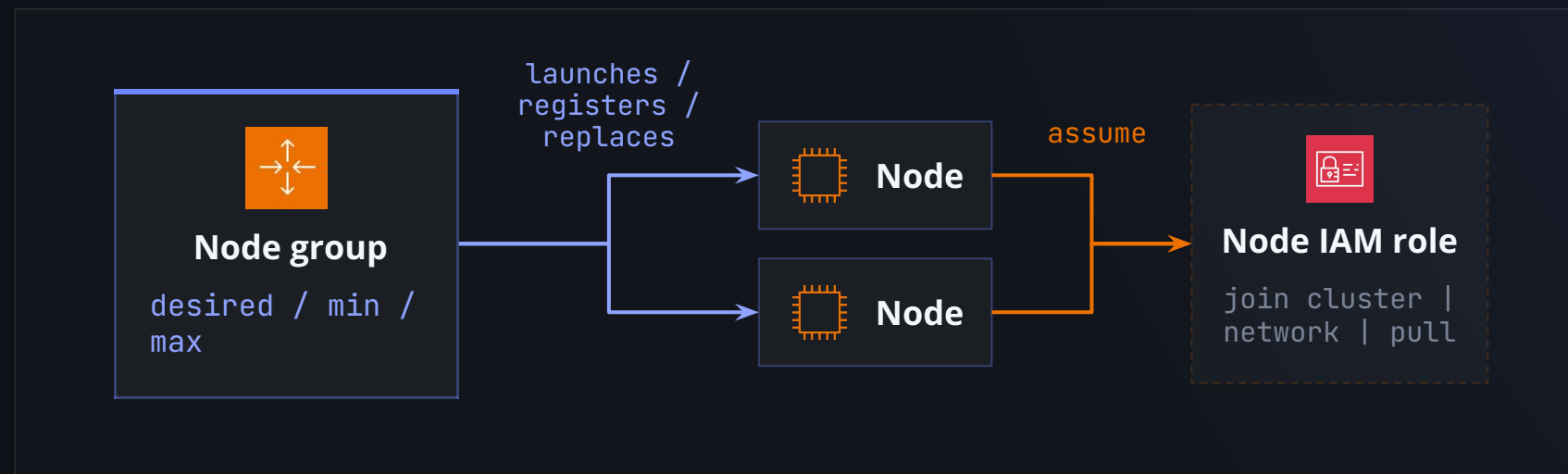


Because the control plane creates real resources in your account, it must assume an IAM role. **This is the first of the two roles.**

WORKER NODES

Worker nodes: EC2 in a node group

You declare how many nodes and which instance types. The node group launches them, registers them with the control plane, and replaces failed ones. Each node assumes the node IAM role to function.



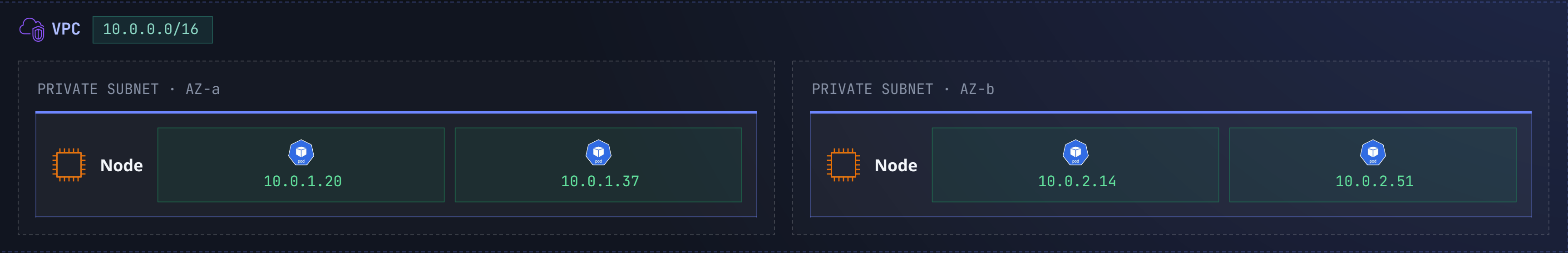
The node IAM role lets each node

- ✓ **Join the cluster.** Register with the control plane's API as a node.
- ✓ **Set up pod networking.** Attach network interfaces and assign pod IPs from the VPC.
- ✓ **Pull container images.** Read from Amazon ECR and other registries.

 Same structure as the control plane: a cluster component that assumes an IAM role to do its job. **This is the second of the two roles.**

Pods get VPC IPs; subnet size caps pods per node

The cluster runs in a VPC with private subnets across two availability zones. The networking addon assigns each pod an address from those subnets, which turns subnet sizing into a real capacity limit.



Two availability zones

A single zone failure removes some capacity, not the cluster.

Private subnets

Nodes have no direct inbound route from the internet, by design.

Pods carry VPC IPs

The subnet's address count is the ceiling on pods per node.

IAM: service roles and operator access

These are constantly confused. One governs what the cluster's own components may do in AWS; the other governs who may call the Kubernetes API.

Component identity

What the cluster may do in AWS

 **Cluster role.** Assumed by the control plane, to create necessary resources within the account.

 **Node role.** Assumed by the nodes, to join, network pods, and pull images.

Both are IAM roles, each with one job.

Operator access

Who may call the Kubernetes API

 **Existing AWS identities.** Operators and pipelines authenticate as the IAM identity they already have.

 **No second credential set.** There is no separate set of Kubernetes credentials to manage.

The AWS identity you already use is what the cluster authorizes.

Addons: pod networking, routing, DNS

A control plane with nodes attached cannot run real workloads until these exist. Their install order is fixed by dependencies.

Addon	What it provides	When it must be installed
vpc-cni pod networking	Assigns each pod a VPC IP address.	before nodes Ready
kube-proxy service routing	Forwards traffic to Services inside the cluster.	alongside nodes
coredns in-cluster DNS	Lets pods find one another by name.	after nodes running

note EKS manages these as versioned addons, so you do not maintain them by hand.